Львівський національний університет імені Івана Франка · Факультет електроніки та комп'ютерних технологій · Кафедра системного проєктування

Сховища знань та управління знаннями · 2026–2027 н. р.

Лабораторна робота 8

RDF-сховище та SPARQL-запити

- **Тема:** ЛР 8. RDF-сховище та SPARQL-запити
- **Лекція:** 8-й тиждень, лекція 8 «RDF, RDFS, OWL, SHACL і SPARQL»
- **Тривалість:** 3 акад. год аудиторно + ≈ 4 год самостійної підготовки й оформлення
- **Оцінювання:** максимум 3 бали = 5 результатів × 0.5 (разом 2.5) + 0.5 бала за захист (5 запитань × 0.1)
- **Термін здачі та захисту:** до кінця 8-го тижня
- **Інструменти:** rdflib, Apache Jena/GraphDB, pySHACL
- **Вхід:** онтологія та індивіди з ЛР 7; Apache Jena Fuseki або GraphDB Free; rdflib, pySHACL

1. Мета роботи

Експортувати знання в RDF, завантажити в триплстор, виконати SPARQL-запити до CQ, додати SHACL-форми з навмисними дефектами та оцінити text-to-SPARQL від LLM із перевіркою за схемою.

2. Індустріальний контекст і роль LLM-експерта

Ви працюєте над власною індустріальною проблемою (тематична область з переліку) у проєкті Domain Knowledge Hub. Генеративну модель ви використовуєте як експерта: вона пропонує знання та чернетки, а ви їх **верифікуєте** і лише потім формалізуєте.

Роль: SPARQL-експерт, схема графа в підказці. Конвеєр: парсер → перевірка схеми → виконання з LIMIT. Рівні: V2 (парсер, схема), V4 (метрики на 15 питаннях). Метрики: частка синтаксично коректних, «лише зі схеми», з правильним результатом.

3. Які навички ви наберете

- працювати з Turtle/JSON-LD і триплстором;
- писати SPARQL (SELECT, ASK, CONSTRUCT, агрегації, property paths, federated);
- описувати й застосовувати SHACL-форми;
- будувати та оцінювати text-to-SPARQL.

4. Підготовка

- Прочитайте лекцію 8.
- Розгорніть Fuseki/GraphDB (Docker або локально).
- Підготуйте онтологію та CQ з ЛР 4/7.

5. Порядок виконання

Робота складається з п’яти результатів. Кожен результат — самостійна частина з конкретним **артефактом** (файл у репозиторії Hub) і чотирма перевірюваними критеріями. Виконуйте результати послідовно: кожен наступний використовує попередній.

Як оцінюється результат. Кожен результат має артефакт і 4 критерії перевірки (C1–C4; C1 — ключовий). 0.5 бала — виконано всі 4 критерії; 0.25 бала — виконано C1 і ще щонайменше один критерій; 0 — інакше. Критерії бінарні («є / немає») і перевіряються за артефактом, тому ви можете самостійно оцінити роботу до здачі.

Результат 1. RDF-дані у триплстоге

Артефакт: data/kb.ttl; docs/load_log.md

Крок 1. Експортуйте онтологію та індивіди в Turtle; збагатіть даними (≥ 300 триплетів).

Крок 2. Завантажте у триплстор (dataset); налаштуйте префікси.

Крок 3. Перевірте кількість триплетів запитом COUNT; запишіть версію й дату.

Крок 4. Збережіть JSON-LD версію ≥ 20 сутностей.

№	Критерій перевірки	✓
C1	≥ 300 триплетів, граф валідний (rdflib).	☐
C2	Завантаження підтверджено запитом COUNT.	☐
C3	Префікси узгоджено; є версія/дата завантаження.	☐
C4	Є JSON-LD для ≥ 20 сутностей.	☐

Результат 2. SPARQL-запити до CQ

Артефакт: queries/*.rq; docs/sparql_results.md

Крок 1. Напишіть ≥ 8 запитів, що відповідають CQ: SELECT, ASK, CONSTRUCT, агрегація, OPTIONAL, FILTER, property path, federated до Wikidata.

Крок 2. Для кожного: CQ, запит, результат, інтерпретація.

Крок 3. Оптимізуйте один повільний запит і порівняйте час.

Крок 4. Перевірте результати вручну на ≥ 3 запитах.

№	Критерій перевірки	✓
C1	≥ 8 запитів, є всі зазначені конструкції.	☐
C2	Кожен запит має зв’язок із CQ і інтерпретацію.	☐
C3	Є порівняння часу для оптимізованого запиту.	☐
C4	≥ 3 запити перевірено вручну (відповідність очікуванню).	☐

Результат 3. SHACL-валідація

Артефакт: shapes/shapes.ttl; docs/shacl_report.md

Крок 1. Опишіть ≥ 5 форм (≥ 8 обмежень): minCount, datatype, class, діапазон, pattern.

Крок 2. Створіть валідний набір і набір з 3 навмисними дефектами.

Крок 3. Запустіть pySHACL; збережіть звіти.

Крок 4. Поясніть, чому SHACL, а не OWL, відповідає за ці обмеження.

№	Критерій перевірки	✓
C1	≥ 5 форм, ≥ 8 обмежень різних типів.	☐

C2	Валідний набір проходить; дефектний — ≥ 3 порушення виявлено.	☐
C3	Звіти збережено й прокоментовано.	☐
C4	Є пояснення різниці SHACL/OWL (≥ 3 речення).	☐

Результат 4. Text-to-SPARQL з оцінкою

Артефакт: tests/t2s.jsonl; docs/t2s_eval.md

Крок 1. Складіть ≥ 15 питань (різної складності) та еталонні запити/відповіді.

Крок 2. Реалізуйте конвеєр: підказка зі схемою → LLM → парсер → перевірка схеми → виконання з LIMIT.

Крок 3. Обчисліть: частку коректних синтаксично, «лише зі схеми», з правильним результатом.

Крок 4. Проаналізуйте 5 помилок; покращіть підказку й повторіть; покажіть до/після.

№	Критерій перевірки	✓
C1	≥ 15 питань з еталоном.	☐
C2	Конвеєр містить парсер і перевірку схеми.	☐
C3	Три метрики обчислено до та після покращення.	☐
C4	Є аналіз ≥ 5 помилок із класифікацією.	☐

Результат 5. Зв'язки з відкритими графами

Артефакт: data/links.ttl; docs/links_verification.md

Крок 1. Знайдіть для ≥ 5 понять відповідники у Wikidata/DBpedia/CWE (за назвою + атрибутами).

Крок 2. Запишіть skos:exactMatch/closeMatch або owl:sameAs з коментарем про ступінь впевненості.

Крок 3. Верифікуйте кожен зв'язок (V1) і за потреби відхиліть.

Крок 4. Виконайте federated-запит, що використовує ці зв'язки.

№	Критерій перевірки	✓
C1	≥ 5 зв'язків із зовнішніми графами.	☐
C2	Кожен має обґрунтування й рівень впевненості.	☐
C3	Є хоч 1 відхилений зв'язок із поясненням.	☐
C4	Federated-запит працює (запит і результат).	☐

6. Обов'язкові артефакти протоколу «LLM-експерт»

У будь-якій ЛР, де ви використовуєте LLM, до звіту додаються:

- Журнал взаємодії з LLM (llm_log/): дата, модель і версія, параметри (temperature, seed), версія підказки, сирий вихід, рішення людини.
- Таблиця тверджень (claims/claims.csv): id, твердження, тип (факт/правило/гіпотеза), джерело, рівень верифікації (V1–V4), статус (verified/rejected/pending), коментар.
- Частка підтверджених тверджень, кількість відхилених із поясненням, перелік джерел.

Без цих артефактів результат, що спирається на LLM, не може отримати 0.5 бала (максимум 0.25).

7. Вимоги до звіту та файлів

- Звіт за шаблоном «Шаблон звіту до лабораторної роботи»; для кожного результату — розділ із кроками, ключовими фрагментами коду/таблицями, рисунками та висновком.
- Код і дані — у репозиторії Hub (гілка lab08); запуск з нуля описано в README (версії, залежності, команди, seed).
- Ключі API не зберігаються в коді та звіті; персональні й конфіденційні дані не передаються в зовнішні сервіси.
- Усі джерела вказано повно; рисунки й таблиці підписано.

8. Оцінювання

- **Результати 1–5:** по 0.5 бала (максимум 2.5): 0.5 — усі критерії C1–C4; 0.25 — C1 + ще хоч один; 0 — інакше.
- **Захист:** 0.5 бала — 5 запитань по 0.1 бала (правильна й самостійна відповідь). Студент також має пояснити будь-яку частину роботи й за вимогою змінити код/запит «наживо».
- **Коригувальні правила:** запізнення (до закриття модуля) — бали за результати × 0.8; «не пояснив роботу» — бали за результати × 0.5; порушення доброчесності — 0 балів за ЛР. Підсумок за ЛР — від 0 до 3 балів.

9. Контрольні питання (захист)

1. Що таке RDF-триплет і яка роль IRI?
2. Чим SHACL відрізняється від OWL?
3. Поясніть один із ваших SPARQL-запитів (наприклад, з property path).
4. Як ви перевіряли запити від LLM?
5. Який із зв’язків з Wikidata ви відхилили і чому?

10. Академічна доброчесність та використання генеративного ШІ

Використання генеративного ШІ дозволено й очікується — це частина методу курсу. Умови: (1) декларація в звіті (інструмент, призначення, приклад запиту, як перевірено); (2) повний журнал взаємодії; (3) ви відповідаєте за кожне твердження та розумієте його. Вигадані дані чи джерела, неоголошене використання ШІ, спільне виконання без декларації — порушення доброчесності.

11. Рекомендовані джерела

- Лекція 8; W3C RDF 1.1, SPARQL 1.1, SHACL; DuCharme (2013); документація rdflib/pySHACL.